Lab 1.
Setting up a development environment for HTML, CSS, and JavaScript/TypeScript

> **Note.** This document was prepared as of September 10, 2026. Menu names, IDE versions, plugins, and AI features may change; if anything differs, refer to the product's current official documentation.

1. Aim

Set up a working local frontend development environment for upcoming HTML, CSS, and JavaScript/TypeScript labs: a code editor or IDE, code quality tools, local web page preview, Git/GitHub, publishing through GitHub Pages and Vercel, and controlled use of an AI agent.

Expected outcome: You can open a project, make a change, check it locally, save it in Git, update the GitHub repository, and confirm that the change has been published to GitHub Pages and Vercel.

2. Objectives

- Explore the roles of HTML, CSS, JavaScript, and TypeScript in a modern frontend project, and learn where to find standards and practical documentation.
- Compare VS Code, WebStorm, Cursor, and Windsurf in terms of web technology support, extensibility, Git, debugging, and AI capabilities.
- Install a minimal set of tools for formatting, static analysis, and local preview.
- Configure an AI agent using rules/instructions, repeatable procedures, and effective prompts.
- Create or verify a GitHub account, repository, and GitHub Pages site.
- Create or verify a Vercel account and make the first deployment from GitHub.
- Build the habit of checking AI-generated changes, linter results, the diff, and the page's actual behavior before committing and pushing.

3. Background

3.1. HTML, CSS, JavaScript, and TypeScript: what are we setting up?

HTML describes document structure and content semantics; CSS handles presentation and layout; JavaScript adds behavior and program logic in the browser; TypeScript extends JavaScript with static typing and compiles to JavaScript. It is important to distinguish language standards from the tools that help you write, check, and run code.

The official HTML Living Standard is a living WHATWG specification with a separate "Edition for Web Developers." The W3C describes the CSS ecosystem, including CSS Snapshot 2026, which summarizes the current state of CSS modules. JavaScript is formally standardized by Ecma International in ECMA-262; MDN is the most convenient reference for practical work with the language. TypeScript has its own Handbook and Reference, with the Handbook intended as the primary day-to-day learning resource.

Technology	Main official source	What it provides	Does it describe IDE setup?
HTML	WHATWG HTML Living Standard	Semantics, document structure, the DOM, and browser algorithms	No. It is primarily a specification; setup is covered by tool documentation.
CSS	W3C CSS / CSS Snapshot 2026	CSS modules, syntax, the cascade, layout, and compatibility	Minimal; authoring tools are mentioned in general terms.
JavaScript	ECMAScript + MDN	Language constructs, APIs, and practical examples	MDN has a separate Environment Setup module, but it does not document a specific IDE.
TypeScript	TypeScript Handbook / Reference	Typing, compiler options, everyday types, and modules	Covers tooling, but IDE setup is detailed in the IDE's documentation.

3.2. Where official documentation covers IDEs and environment setup

The documentation can usefully be divided into three levels. First are standards: they define a language or platform, not a particular editor. Second are learning resources such as MDN and TypeScript, which begin to recommend editors, terminals, browsers, and local environments. Third is the documentation for a specific IDE, which covers settings, plugins/extensions, linting, debugging, previews, workspace configuration, and AI.

- MDN has a dedicated Environment Setup module covering text editors, browsers, the file system, and the command line. It is the most direct official resource for an initial learning setup.
- VS Code has separate official pages for HTML, CSS/SCSS/Less, and TypeScript, including IntelliSense, formatting, debugging, and extensions.
- WebStorm documents global and project-level settings, plugins, HTML/CSS/JS/TS support, and ESLint/Prettier integration.
- Cursor and Windsurf are positioned as AI-first environments, so their documentation adds Agent/Cascade, rules, skills, context, and other AI components on top of the familiar IDE model.

3.3. Comparing IDEs and editors

Criterion	VS Code	WebStorm	Cursor	Windsurf
Positioning	Lightweight, extensible editor with IDE-like features	Full-featured JetBrains IDE for JavaScript/TypeScript/web	VS Code-based AI-first editor	VS Code-based AI-first editor
HTML/CSS/JS/TS	Strong built-in support	Deep built-in support	Similar to VS Code, plus AI	Similar to VS Code, plus AI
Extensions	Visual Studio Marketplace	JetBrains Marketplace + bundled plugins	Open VSX + first-party/audited replacements for some extensions	Open VSX / compatible plugins, depending on version
Git	Built-in + extensions	Deep built-in integration	Built-in	Built-in
Debugging	JavaScript/TypeScript/Node.js	Client- and server-side JS, test integration	Through the VS Code stack + Agent	Through the VS Code stack + Cascade
AI	GitHub Copilot / other plugins	Junie / AI Assistant and other plugins	Agent, rules, skills, MCP, etc.	Cascade, rules/memories/workflows/skills, as described in current documentation
Course recommendation	Primary baseline	Strong alternative	AI-first alternative	AI-first alternative

3.4. Formatting, linting, and local preview

Prettier and ESLint solve different problems. Prettier normalizes code formatting; ESLint analyzes code against quality and style rules and checks for potential errors. It is useful to learn both kinds of tools: formatting does not mean the code is correct, and linting should not become manual formatting of every space.

Static HTML/CSS/JS projects need a local server/preview so you can check browser behavior in conditions close to a real web environment. Microsoft Live Preview provides a local server and live reload; WebStorm has its own preview and web tooling.

3.5. Git, GitHub, GitHub Pages, and Vercel

Git stores local change history; GitHub provides a remote repository and a shared space for code. GitHub Pages publishes a static site without separate hosting, while Vercel can be connected to a Git repository and automatically start a deployment after a push. This lets you complete the full cycle in one lab: edit → verify → commit → push → deployment.

- **GitHub Pages:** for a basic setup, a repository with an `index.html` entry file and Pages configured to publish from a branch/directory is sufficient.
- **Vercel:** importing a repository creates a project; pushing to the connected Git repository starts a new deployment, and a PR can have a preview deployment.

3.6. AI agents: chat, agents, rules, and skills

In modern IDEs, AI features are more than autocomplete. An AI agent can search for files, analyze a repository, edit multiple files, and run commands and tests. You therefore need to learn not only how to write a prompt, but also how to set an agent's autonomy boundaries, review its diff, and distinguish "project context" from "instructions for a specific step."

Mechanism	Purpose	Example	When to use
Prompt	One-off task	"Find and fix the cause of the 404"	One specific task
Rule / AGENTS.md	Persistent project rules	"Use TypeScript; run lint before committing"	Shared or ongoing requirements
Skill (SKILL.md)	Repeatable procedure + knowledge	"Review the frontend using an accessibility checklist"	A complex but common workflow
Workflow / command	Manually launched sequence	"/deploy-preview"	When a procedure should run explicitly
MCP / tools	Access to external context or services	Documentation, issue tracker	When external data/actions are needed

Cursor officially supports Agent Skills as an open standard, project rules, and `AGENTS.md`. The Agent Skills specification describes a portable approach: a directory containing `SKILL.md`, which may include instructions, scripts, references, and assets. GitHub Copilot supports repository-wide instructions in `.github/copilot-instructions.md` and `AGENTS.md` for agent instructions; Junie also uses `AGENTS.md`. Current Windsurf documentation describes agent rules, memory, and repeatable workflows/skills; specific menu names may change.

3.7. Writing prompts

An effective coding agent prompt should provide more than "please make it look nice": it should include context, the desired outcome, constraints, and verification criteria. It is especially useful to distinguish intent context (what

you want) from state context (what is currently in the project): a file, error, stack trace, screenshots, browser requirements, and so on.

- First, ask the agent to briefly describe its plan and the files it intends to change.
- For a small lab, give it one functional task per iteration.
- Specify completion criteria: lint passes, the page opens, the button works, and the mobile viewport has been checked.
- Ask it not to change configuration or dependencies without explaining why.
- After a change, review the diff and run local checks before committing and pushing.

3.8. Academic integrity and safe use of AI

An AI agent is a developer support tool, not a source of truth. Its suggestions may be incomplete, outdated, or unsuitable for a particular project. Do not send secrets, tokens, passwords, private keys, or personal data to an AI tool unless the tool's policy and settings explicitly permit it. You must be able to explain the changes you accepted into your code and demonstrate that you tested them.

4. Lab assignment

1. Review the HTML/CSS/JavaScript/TypeScript documentation about development environments and identify which recommendations concern standards and which concern IDEs/tooling.
2. Install and configure one of the recommended IDEs: VS Code, WebStorm, Cursor, or Windsurf.
3. Check for HTML, CSS, JavaScript, TypeScript, and Git support, as well as browser debugging/preview.
4. Install third-party tools for the chosen IDE (at least ESLint + Prettier; for an editor without a built-in preview, Live Preview or an equivalent).
5. Create a small static web project with `index.html`, `styles.css`, `script.js` or `script.ts`, and `README.md`.
6. Configure an AI agent: create `AGENTS.md` or equivalent project rules; if possible, create one Skill with a `SKILL.md`.
7. Create a GitHub repository and make the first commit and push.
8. Enable GitHub Pages and obtain a public URL.
9. Connect the repository to Vercel and obtain a deployment URL.
10. Make a small change to the page, check it locally, commit and push it, and confirm that GitHub Pages and Vercel have updated.
11. Prepare a report that meets the requirements in Section 6.

5. Procedure

5.1. Preparing the development environment

1. Install Git and Node.js LTS. At the time this document was prepared, the current Node.js LTS line is 24.x; get the specific patch release from the official download page.

   ```
   git --version
   node --version
   npm --version
   ```

2. Install a modern browser with Developer Tools. Chrome or Edge is recommended, along with a second browser for basic cross-browser checks.

3. Create a GitHub account and sign in. For this lab, the instructor must be able to access the repository using the link you provide.
4. Create a Vercel account and, if possible, link it to GitHub.

5.2. Configuring VS Code (recommended baseline)

1. Install Visual Studio Code from the official website.
2. Open the root folder of your future project using **File → Open Folder**.
3. Confirm that HTML, CSS, and JavaScript/TypeScript are recognized as the corresponding languages and that IntelliSense/Emmet work.
4. Install the ESLint, Prettier - Code formatter, and Live Preview extensions. GitLens and Error Lens are optional.
5. Create `.vscode/extensions.json` with workspace extension recommendations.

```
{
  "recommendations": [
    "dbaeumer.vscode-eslint",
    "esbenp.prettier-vscode",
    "ms-vscode.live-server"
  ]
}
```

6. Create `.vscode/settings.json` and set the basic options.

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },
  "editor.tabSize": 2,
  "editor.insertSpaces": true,
  "files.trimTrailingWhitespace": true,
  "files.insertFinalNewline": true
}
```

7. Create `.prettierrc.json` so formatting is consistent across IDEs.

```
{
  "printWidth": 100,
  "tabWidth": 2,
  "useTabs": false,
  "semi": true,
  "singleQuote": true
}
```

8. Create an ESLint configuration for JavaScript/TypeScript that matches the current ESLint version. For ESLint 9, the recommended formats are `eslint.config.js` / `.mjs` / `.cjs` / `.ts` / `.mts`.

5.3. Configuring WebStorm

1. Install WebStorm and open the project's root folder.
2. Review **Settings → Plugins**, as well as the Languages & Frameworks and Editor settings.
3. Confirm that HTML, CSS, JavaScript, and TypeScript are supported without additional language packages.
4. Enable/configure ESLint for the project. WebStorm displays lint warnings and errors directly in the editor.

5. If needed, install/configure Prettier through JetBrains Marketplace or the project's `package.json`.
6. Configure formatting using one shared `.prettierrc.json` so different IDEs do not apply different formatting rules.

5.4. Configuring Cursor

1. Install Cursor, sign in, and open the project.
2. If you are switching from VS Code, check that your VS Code settings and extensions were imported.
3. Confirm that the AI Agent can see the repository root and find HTML/CSS/JS/TS files.
4. Create `AGENTS.md` in the project root for general rules.
5. Create `.agents/skills/frontend-review/SKILL.md` or `.cursor/skills/frontend-review/SKILL.md`. `.agents/skills` is recommended as a more portable option for compatible agents.

5.5. Configuring Windsurf

1. Install Windsurf, complete onboarding, and open the project root.
2. If appropriate, check that your configuration from VS Code or Cursor was imported.
3. Review Extensions/Plugins and install only the frontend tools missing from your Windsurf installation.
4. Configure Cascade: rules/instructions, repeatable workflows/skills, and restrictions on file changes.
5. Confirm that the AI agent can use the terminal, but review its commands before accepting changes, especially commands that delete files or change dependencies.

5.6. Creating a practice web project

1. Create this structure:

```
frontend-lab-1/
├── index.html
├── styles.css
├── script.js
├── README.md
├── .gitignore
├── .editorconfig
├── .prettierrc.json
├── .vscode/
│   ├── settings.json
│   └── extensions.json
├── .github/
│   └── copilot-instructions.md   # if using Copilot
└── AGENTS.md                     # general AI instructions
```

2. In `index.html`, build a simple page with a heading, paragraph, button, information block, and footer.
3. In `styles.css`, create a responsive layout for desktop and mobile viewports.
4. In `script.js`, implement a minimal interaction: change text, toggle a class, switch the theme, or add a counter.
5. Add a `README.md` with the project name, a short description, instructions for running it locally, and GitHub Pages/Vercel links after deployment.

5.7. Configuring an AI agent

1. Create `AGENTS.md`.

```
# Frontend Lab Agent Instructions

## Role
Act as a careful frontend development assistant for a 3rd-year CS student.

## Project
- HTML + CSS + JavaScript/TypeScript.
- Prefer browser-native APIs and simple solutions.
- Do not add dependencies unless there is a clear reason.

## Before editing
1. Inspect the relevant files.
2. State a short plan.
3. Identify risks or missing context.

## After editing
1. Check the diff.
2. Run the relevant lint/format/type checks.
3. Report exactly what changed and what was verified.

## Safety
Never expose or invent secrets. Do not delete files or rewrite project
configuration without explaining why.
```

2. Create one Skill, for example `frontend-review`.

```
---
name: frontend-review
description: Review a small frontend change for correctness, accessibility, responsiveness,
and maintainability. Use when the student asks for a pre-commit review or before deployment.
---

# Frontend Review

## Checklist
1. Inspect only the files relevant to the task.
2. Check HTML semantics and obvious accessibility issues.
3. Check CSS responsiveness and class naming consistency.
4. Check JavaScript/TypeScript for runtime errors and unnecessary complexity.
5. Run the project's available lint/type checks.
6. Summarize findings by severity: blocker, warning, suggestion.
7. Do not make changes unless explicitly asked or the agent workflow permits fixes.
```

3. Check that the agent actually follows the rules. Try this test prompt: "Find issues in `index.html` and `styles.css`, but do not change anything; show only a list of issues with explanations."
4. Give it a second prompt for a real change, for example: "Add a dark theme toggle. First show your plan; after making the change, run checks and briefly describe the diff."

5.8. Working with Git and GitHub

1. In the project root, run:

```
git init
git add .
git commit -m "chore: initialize frontend lab 1"
git branch -M main
```

2. On GitHub, create an empty repository without an automatically generated README if your local project already has its own README.
3. Add a remote and push the code:

```
git remote add origin https://github.com/<username>/<repository>.git
git push -u origin main
```

4. Check the repository in a browser. It should contain the source files, IDE configuration, and AI instructions, but not `node_modules`, secrets, local caches, or operating system files.

5.9. GitHub Pages

1. Open GitHub → repository → **Settings** → **Pages**.
2. Under **Build and deployment**, select **Source** → **Deploy from a branch**.
3. Select the `main` branch and the `/(root)` folder if `index.html` is in the repository root.
4. Wait for the site to be published and open its GitHub Pages URL. Record it for your report.
5. If you get a 404, first check that `index.html` is at the top level of the selected publishing source.

5.10. Vercel

1. Sign in to Vercel and select **New Project / Import Git Repository**.
2. Give Vercel access to the required GitHub repository and select it.
3. For a simple static project, keep the automatically detected settings if they are correct. Check the root directory and build settings before deploying.
4. Start the deployment and open the resulting deployment URL.
5. After your next `git push`, check that Vercel created a new deployment. If possible, record the deployment URL in the README.

5.11. Test change and final verification

1. Change one UI element, such as a button label, color, theme toggle, or a new section.
2. Start the local preview and check both desktop and mobile viewports.
3. Run formatting and linting. If using TypeScript, check for type errors.
4. Review `git diff` and confirm that only the intended files will be committed.
5. Create a commit:

```
git add .
git commit -m "feat: update landing page interaction"
git push
```

6. After the update, check the GitHub repository, GitHub Pages, and Vercel.

6. Report requirements

The report should be concise, evidence-based, and reproducible. It should show not merely that "the IDE is installed," but that the environment actually works and that you can complete the full change-and-publishing cycle.

6.1. Sample email

Subject: `[Web] Lab 1: Development Environment Setup`

Include:

- GitHub repository: `https://github.com/<username>/<repository>`
- GitHub Pages: `https://<username>.github.io/<repository>/`
- Vercel: `https://<project>.vercel.app/`
- The IDE you chose.
- A list of key installed extensions/plugins and their purposes.
- The results of checking Git, Node.js, and npm (versions).
- A description of the configured AI agent: which agent you used and where its rules are stored.
- A brief conclusion about the lab.

6.2. Sample report structure

Section	Contents
1. Topic and aim	Lab title, aim, full name, group
2. Environment	IDE, OS, Git, Node.js, key plugins
3. AI agent	Agent, AGENTS.md/rules, skill, 2–3 example prompts
4. GitHub	Repository, commit/push, link
5. Deployment	GitHub Pages and Vercel links
6. Test change	What changed, how it was checked, and how deployments were updated
7. Conclusion	What was configured and checked, and what was challenging

7. Review questions

1. How do the HTML Living Standard, CSS specifications, the ECMAScript specification, and the TypeScript Handbook differ?
2. Why does a language standard not specify a particular IDE?
3. What is the difference between a formatter and a linter?
4. Why should `.prettierrc` and IDE settings be complemented by project configuration?
5. What are the advantages and disadvantages of VS Code compared with WebStorm for a student?
6. What is the fundamental difference between a regular chat assistant and a coding agent?
7. When should you use `AGENTS.md`/rules, and when should you use a Skill?
8. What should you check before accepting AI-generated changes?
9. What triggers a new Vercel deployment when a Git repository is connected?
10. Why is an `index.html` file important for a basic GitHub Pages deployment?

8. Lab completion criteria

Criterion	Minimum requirement
IDE	Installed and opens the project; HTML/CSS/JS/TS features work
Tooling	ESLint + Prettier are configured; local preview/debug is available
AI	Agent is available; `AGENTS.md` or equivalent rules are created; one reusable skill is created if the tool supports it
GitHub	Repository is created, code is pushed, and there are at least 2 meaningful commits
GitHub Pages	The public URL opens the latest version of the site
Vercel	Deployment succeeds; a new version is created after a push
Change	You demonstrate edit → verify → commit → push → deploy
Report	Includes links, a brief description of the environment and AI setup, and a conclusion

9. References

The following are primarily official sources from product vendors and standards organizations. Give them priority over third-party articles and videos while completing the lab.

1. WHATWG. HTML: The Living Standard — Edition for Web Developers
2. W3C. Cascading Style Sheets home page
3. W3C. CSS Snapshot 2026
4. ECMA International / TC39. ECMAScript 2025 Language Specification
5. MDN Web Docs. Web developer guides
6. MDN Web Docs. Environment Setup
7. MDN Web Docs. JavaScript Guide
8. MDN Web Docs. CSS — Getting started
9. TypeScript. The TypeScript Handbook
10. Visual Studio Code. HTML in Visual Studio Code
11. Visual Studio Code. CSS, SCSS and Less
12. Visual Studio Code. TypeScript in Visual Studio Code
13. Visual Studio Code. Extension Marketplace
14. Visual Studio Marketplace. ESLint
15. Visual Studio Marketplace. Prettier - Code formatter
16. Microsoft / VS Code. Live Preview
17. JetBrains. Configure WebStorm settings
18. JetBrains. HTML in WebStorm
19. JetBrains. CSS in WebStorm
20. JetBrains. JavaScript in WebStorm
21. JetBrains. TypeScript in WebStorm
22. JetBrains. Plugins in WebStorm
23. Cursor. Cursor — Installation
24. Cursor. Agent overview
25. Cursor. Rules
26. Cursor. Agent Skills
27. Cursor. Working with Context

28. GitHub. Adding repository custom instructions for GitHub Copilot
29. GitHub. Creating a GitHub Pages site
30. GitHub. Configuring a publishing source for GitHub Pages
31. GitHub. Creating a repository for your project
32. Vercel. Deploying Git Repositories with Vercel
33. Vercel. Deploying to Vercel
34. Node.js. Download Node.js
35. npm. Downloading and installing Node.js and npm
36. Agent Skills. Agent Skills Overview
37. Agent Skills / GitHub. Specification and documentation for Agent Skills
38. Anthropic. Prompting best practices
39. Windsurf / Devin Docs. Plugins / Windsurf
40. Windsurf / Devin Docs. Workflows
41. Windsurf / Devin Docs. Cascade overview